Review

Measuring End to End Latency in Robotic Edge Vision

A Critical Review of Timing Boundaries and Public Deployment Evidence

Xiongyule Zhou

School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing 100044, China

* Correspondence: jensenzhou47@gmail.com

Abstract

Robotic perception depends on when a visual result becomes usable, yet deployment papers measure latency across different portions of the sensing and computing pipeline. This critical review examines how those differences constrain comparisons of robotic edge vision. Targeted searches and citation following identify public deployment reports, benchmarking studies, and official timing documentation, with a quantitative evidence core of ten reports. Results are organized by measurement boundary, pipeline stage, and operating condition rather than by detector family. Four boundary classes distinguish engine execution, host processing, acquisition-to-result delay, and physical-event response. Published examples show that acquisition and buffering can outweigh inference, while region-dependent processing and output generation can alter the dominant bottleneck. Several reporting ambiguities prevent apparently precise deployment results from supporting hardware rankings or control deadlines. We therefore introduce a four-gate comparison procedure that checks boundary, workload, operating point, and statistic before interpreting a speedup. A minimum reporting specification links frame provenance, clock semantics, queue policy, stage timing, tail behavior, and task quality. The synthesis supports conditional optimization of the measured bottleneck, but does not establish a universal platform ranking or a safety guarantee. Its practical contribution is an auditable route from published timing values to bounded deployment conclusions.

Keywords: robotic perception; edge computing; end-to-end latency; embedded vision; timing measurement; benchmarking; information freshness; inference deployment

1. Introduction

A robot acts on a scene that may have changed while its vision system was processing an image. Consequently, a deployment can produce frequent, accurate detections whose underlying observations are already stale. Visual feedback research establishes the importance of the perception-control relationship, while streaming perception makes the mismatch between offline accuracy and timely output explicit [1, 2]. For mobile manipulation and navigation, the relevant engineering question is therefore when a usable visual result reaches its consumer.

Edge deployment changes that question in several ways. Local computation avoids some network transfers, but shares memory bandwidth, power, and thermal capacity with other robot functions. Remote execution offers additional computing resources while adding compression, communication, scheduling, and response handling. Broad edge-intelligence and efficient-inference surveys explain these trade-offs [3, 4, 5]. They do not make every published timing value interchangeable. A network forward pass, a complete host pipeline, and an image-to-response service can all be described as latency.

This ambiguity survives even when a paper uses the term end-to-end. Jeon et al. explicitly place the precise origin of their detector delay at image arrival in the driver buffer [6]. R-TOD instead defines delay from an object's appearance to its reported detection [7]. These definitions address related engineering problems, but the later study is not simply a replacement measurement of the same interval. A review that combines their values without retaining the origins would obscure the distinction it intends to explain.

Recent reviews already cover real-time vision architectures, compression, accelerators, and deployment workflows. Elhanashi and Saponara connect these topics across application domains and discuss inconsistent benchmarking [8]. Cordova-Cardenas et al. organize embedded deployment into an engineering

workflow [9]. Varghese et al. provide a broader taxonomy of edge performance benchmarking [10]. The additional contribution is a source-level analysis of which timing claims support which robotic deployment conclusions.

The review asks three questions. Which start events and output events are actually represented by public latency reports? How do acquisition, transfer, preprocessing, inference, and output handling change the observed bottleneck? Under what conditions can two reported results support a comparative conclusion? The scope is frame-based robotic edge perception and transferable embedded-vision subsystems. Event cameras and vision-language systems are discussed where they expose the limits of this scope.

Three contributions follow. First, four measurement-boundary classes separate engine execution from host processing, acquisition-to-result delay, and physical-event response. Second, a traceable evidence table preserves deployment settings and unresolved reporting ambiguities alongside the values. Third, a four-gate procedure and reporting specification translate that evidence into conditional comparison rules. These are proposed synthesis tools, not validated standards or new timing experiments. Table 1 positions the review against closely related literature.

Table 1. Position relative to adjacent reviews. The last column describes the additional focus of this review, without implying that the cited reviews ignore timing.

Review	Main organizing question	Additional focus here
Sze et al. [4]; Shuvo et al. [5]	Efficient neural computation and accelerator design	Which portions of a deployed sensing pipeline are timed
Varghese et al. [10]	Systems, methods, and workloads for edge benchmarking	Frame provenance and result age in robotic vision
Cordova-Cardenas et al. [9]	Embedded neural-network deployment workflow	Evidence-level eligibility for numerical comparisons
Elhanashi and Saponara [8]	Architectures, compression, hardware, and application domains	Explicit timing endpoints, ambiguity tracking, and comparison gates

2. Review Design and Evidence Handling

2.1. Scope and source identification

This is a targeted critical narrative review, not a systematic review, scoping review, or meta-analysis. Searches were conducted on 19 October 2026 using web discovery queries, followed by publisher pages, arXiv records, institutional manuscripts, and official project documentation. Query families combined robotic or embedded vision with end-to-end latency, camera buffering, preprocessing, inference benchmarking, Robot Operating System 2 (ROS 2) tracing, offloading, and timing boundaries. Named systems and backward references expanded the search. The supplementary search log records the executed queries and source-retrieval outcomes. No claim of database-wide coverage or screening saturation is made.

The principal deployment window is 2019-2026. Earlier work supplies control and computational context. A quantitative report was eligible when it exposed a measured deployment result or stage-level timing relevant to visual perception on embedded, onboard, near-edge, or robot-connected infrastructure. Its original methods or results had to be publicly inspectable. Papers about gaze estimation and infrastructure inspection were retained as transferable subsystem evidence, rather than relabeled as robot control experiments. Architecture papers supplied context when their measurement scope clarified a deployment issue.

Unverifiable numerical claims, vendor peak-operation ratings without an application measurement, and speedups recoverable only from another review were not used as quantitative evidence. Secondary reviews guided positioning and source discovery. Official documentation supported the interpretation of application programming interfaces (APIs), timestamps, and middleware policies; it was not treated as an independent hardware-performance experiment. The resulting core comprises ten reports, including a workshop paper

and one 2026 preprint. This count describes the selected evidence set, not a systematic-review screening flow.

2.2. Extraction and verification

For each core report, extraction retained the task, model, hardware, sensor and tensor resolutions where available, precision, runtime, batch or request pattern, clock and boundary description, statistic, source locator, and interpretation limits. Quantitative entries were checked against original methods, tables, figures, or results text. Key tables were also inspected in rendered source PDFs. Bibliographic titles, author lists, publication venues, dates, and identifiers were checked against publisher or registry metadata. A published citation and the particular author-manuscript version used for extraction are recorded separately when necessary.

Missing information is coded NR, meaning not reported in the inspected passages, rather than zero or absent from the implementation. Where a source groups several operations, the aggregate is retained. A residual between total and inference time is not relabeled as acquisition or preprocessing. The evidence package distinguishes a reported value, arithmetic derived from reported values, an interpretation, and a proposed reporting practice. Arithmetic uses the displayed precision of the source, without manufacturing additional significant digits.

Extraction and critical appraisal were performed in one workflow without independent duplicate screening or extraction. Ambiguous definitions were recorded without contacting authors. These choices limit reproducibility of the selection and confidence in fine-grained coding. Heterogeneous measurement boundaries and workloads also preclude a defensible pooled effect estimate. The review synthesizes mechanisms and within-study contrasts, while retaining incompatible results as separate observations. Supplementary files provide the evidence table, search and retrieval records, and extraction notes so readers can audit the main comparisons.

2.3. Appraisal focused on the claim being made

A timing result is appraised for boundary validity, workload fidelity, operating-condition disclosure, statistic definition, and inspectable provenance. These dimensions are not combined into a score. A well-specified engine benchmark can strongly support an engine comparison while offering little evidence about scene-to-command delay. Conversely, a robot demonstration can establish practical interaction while leaving the clock origin ambiguous. The appraisal therefore limits the claim supported by each result rather than labeling an entire paper as high or low quality.

3. What an End to End Timing Claim Measures

3.1. Four boundary classes

A latency value needs both a start event and an end event. We use four descriptive classes to expose common differences. B0 covers execution of an inference engine, with transfer inclusion stated separately. B1 covers an application or host pipeline from an available frame or input buffer to a specified result. B2 starts at an identified sensor-acquisition event and ends when the corresponding result becomes usable. B3 starts at an external physical event and ends at a stated perception or action response. These labels are assigned in this review; they are not the source authors' terminology.

The classes describe boundary information, not a quality ladder. A B0 measurement can be appropriate for compiler optimization. B2 is relevant to image age at a controller input. B3 additionally depends on when the event becomes observable, the sampling policy, and the response criterion. The classes are not automatically nested because display, network acknowledgement, model response, and actuator application are different endpoints. Figure 1 places them around a common visual pipeline.

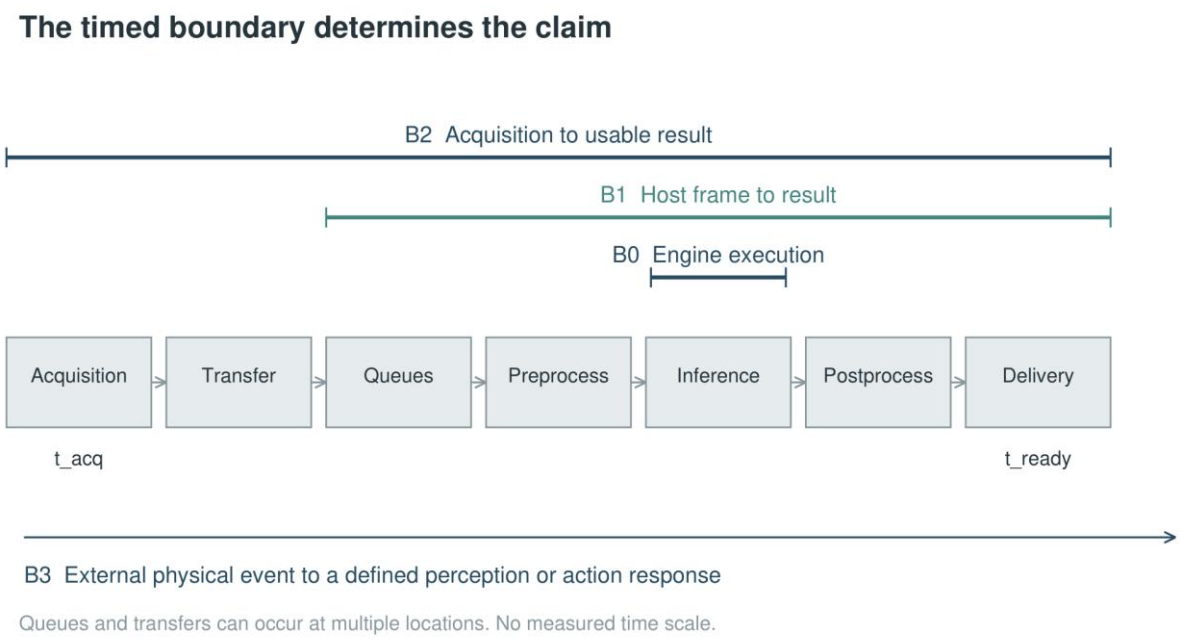


Figure 1. Timing boundaries around a frame-based robotic vision pipeline. B0-B3 are review-assigned descriptions. Transfer may occur before and after inference, and queues may exist between any stages. A physical-event origin also requires a defined response criterion. The diagram is conceptual and has no measured time scale.

For a frame i, let t_acq(i) denote its defined acquisition timestamp and t_ready(i) the instant its usable result reaches the consumer. Its acquisition-to-result latency is

$$L_{frame}(i) = t_{ready}(i) - t_{acq}(i) \quad (1)$$

Acquisition could mean exposure start, exposure midpoint, exposure end, or completed readout. These choices produce different values and should not be silently substituted. The Linux Video for Linux Two interface distinguishes timestamp source flags, including start of exposure and end of frame [11]. The ROS 2 Image message specifies acquisition time for its header, but the message definition alone does not verify a driver's implementation [12]. A host receipt timestamp is therefore insufficient evidence of exposure timing.

3.2. Stage accounting and pipeline overlap

For a single frame following a sequential causal path, its delay can be partitioned into acquisition, transport, queue residence, preprocessing, inference, postprocessing, and delivery. Every partition needs non-overlapping endpoints. The bookkeeping expression is

$$L_{path} = T_{acq} + T_{transfer} + T_{queue} + T_{pre} + T_{infer} + T_{post} + T_{delivery} \quad (2)$$

Equation (2) is not a rule for summing arbitrary profiler outputs. Readout can overlap transfer; a graphics processor can resize one frame while inferring another. Parallel branches require a causal critical path to their merge. If asynchronous operations are timed without completion synchronization, a measured interval may describe submission overhead rather than execution. NVIDIA's documentation distinguishes wall-clock measurement, CUDA events, enqueue overhead, device compute, and host-device copies [13]. The benchmark command, runtime version, and completion event must accompany the number.

Queue time is a residence interval, not an extra computation kernel. It may occur inside camera drivers, decoders, middleware history, inference services, or result consumers. If it is omitted, the stage sum can disagree with an external observation. If it is included in both a host timer and a transport timer, the sum can double count it. A practical extraction table should therefore store the timed interval and its inclusion rules before storing a stage label.

3.3. Latency, throughput, and freshness

Throughput counts completed work per unit time. In a stable serial loop with one frame per cycle, reciprocal throughput approximates the mean cycle interval. It generally does not equal the delay experienced by an

individual frame in a pipeline. The reciprocal of a mean also differs from the mean of reciprocals. Thus, a paper reporting mean latency and mean per-frame FPS may legitimately show non-reciprocal numbers, but it must identify their aggregation before either is used to infer the other.

Freshness introduces a further distinction. Let u(t) be the acquisition time of the newest observation represented by a usable result available at time t. Its information age is

$$A(t) = t - u(t) \quad (3)$$

This adapts the age-of-information perspective to visual observations [14]. Between output updates, age increases even if completed-frame latency is constant. Dropping old frames may improve freshness while reducing the number of processed frames. It may also miss short-lived objects or break temporal tracking assumptions. Streaming perception evaluates predictions against the evolving world and shows why latency and accuracy should be considered jointly [2]. Neither an isolated latency percentile nor an offline accuracy score captures that relationship alone.

3.4. Clocks, tails, and deadlines

Subtracting timestamps from two machines requires a clock relationship, not simply matching units. If the receiver clock has offset delta from the sensor clock, the observed one-way interval includes delta. Clock synchronization uncertainty therefore belongs in the error budget. A round-trip measurement on one monotonic clock avoids this offset subtraction, but includes the return path and server response. It cannot be divided by two without assumptions about path symmetry and processing.

Mean, median, and tail latency answer different questions. The 99th percentile (P99) describes an empirical distribution under a specified workload; it is not a worst-case execution-time guarantee. A sampled maximum also depends on run duration, workload, and censoring. Deadline reporting should identify the deadline, count late or missing results, and state whether timeouts were excluded. MLPerf's separation of benchmark scenarios illustrates why arrival processes and latency constraints belong to the benchmark specification [15]. Robotic timing studies additionally need the sensor and result semantics that an inference benchmark does not prescribe.

4. Deployment Evidence and Bottleneck Mechanisms

4.1. Acquisition and buffering can dominate

Acquisition-related overhead is visible in TinyTracker's in-sensor gaze-estimation deployment [16]. Its 112 × 112 input model has 0.86 ms reported inference time on IMX500, within a 19 ms acquisition-and-inference interval. The reported front end combines pixel reading, processing, and transfer, so its 17.9 ms should not be split into invented subcomponents. Result retrieval adds 0.24 ms. Figure 2 shows this decomposition, and Table 2 retains the corresponding boundary and task qualification.

The example changes the optimization decision. Inference represents approximately 4.5% of the reported total. With every other component fixed, eliminating inference entirely would reduce the total only to approximately 18.1 ms. This is arithmetic from rounded source values, not an observed optimization. The calculation supports investigating the front end in that configuration. It does not show that acquisition dominates all edge vision systems, or that other gaze-estimation hardware preserves identical image quality and timestamp semantics.

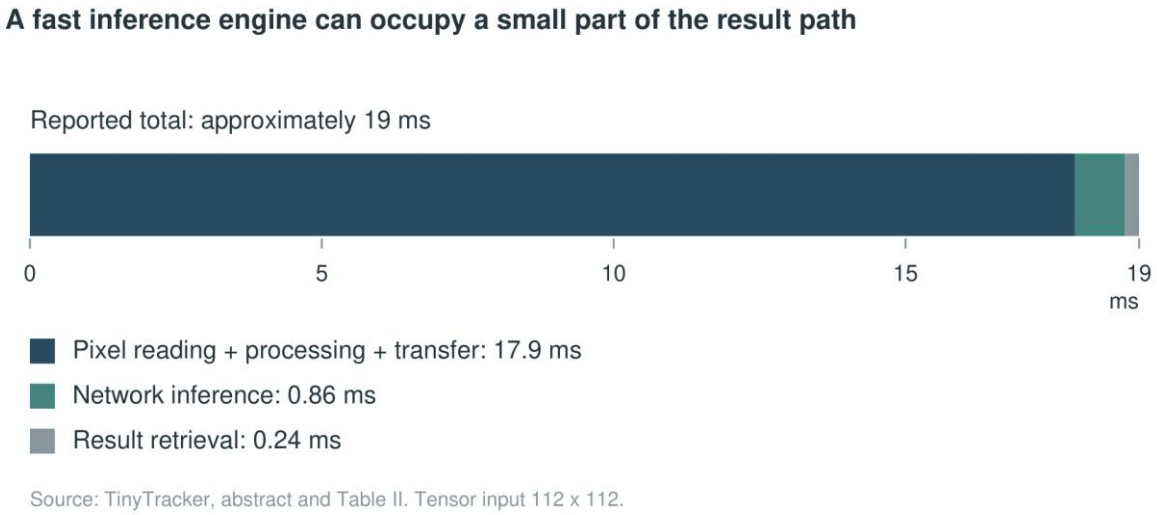


Figure 2. TinyTracker timing decomposition on IMX500, redrawn from the reported values in [16], abstract and Table II. The front-end interval combines pixel reading, processing, and transfer. These components total approximately 19 ms and do not include a measured robot-control response.

Buffering produces a different limitation: work can complete regularly while processing old observations. Jeon et al. change driver buffering and pipeline scheduling without changing detector architecture [6]. Their reported delay reduction is accompanied by a small throughput change, exposing the separation between processing frequency and result age. Their timing origin is explicitly tied to driver-buffer arrival. It does not establish an optical-event-to-controller response.

R-TOD extends this line of work through on-demand capture, reduced pipeline slack, and isolation of competing central processing unit (CPU) and graphics processing unit (GPU) activity [7]. Its within-study progression and tail behavior are shown in Figure 3. Importantly, the camera-subsystem evaluation generates the object-appearance component randomly between capture instants, while measuring transfer. The event-origin interpretation therefore contains a modeled component and should not be described as a wholly optical ground-truth measurement.

Table 2. Acquisition and buffering evidence. FPS means frames per second. The results belong to the stated source configurations and are not a cross-platform ranking. INT8 and FP32 denote 8-bit integer and 32-bit floating-point formats. P99 is the empirical 99th percentile.

Report and setup	Boundary and result	Inference supported
Jeon et al. [6] YOLOv3; Jetson AGX Xavier; input settings NR in the cited timing summary	Driver-buffer arrival to completed display stage 1335 → 438 → 229 ms; 4.48 → 4.51 → 4.30 FPS	Queue and pipeline changes reduce this detector interval; optical acquisition is outside its precise origin
Jang et al. [7] YOLOv3; AGX Xavier; 640 × 480, 30 FPS, YUYV camera; 608 × 608 tensor	Event-origin definition with simulated capture-phase component Mean 1070 → 261 ms; P99 1228 → 410 ms	Within-study scheduling improvements; empirical tails and modeled capture phase do not certify a physical worst case
Bonazzi et al. [16] TinyTracker; IMX500; 112 × 112 tensor; INT8 weights/activations, FP32 outputs	Capture-and-inference interval approximately 19 ms; inference 0.86 ms; front end 17.9 ms; retrieval 0.24 ms	Front-end aggregate dominates this gaze-estimation setup; no robot actuator endpoint

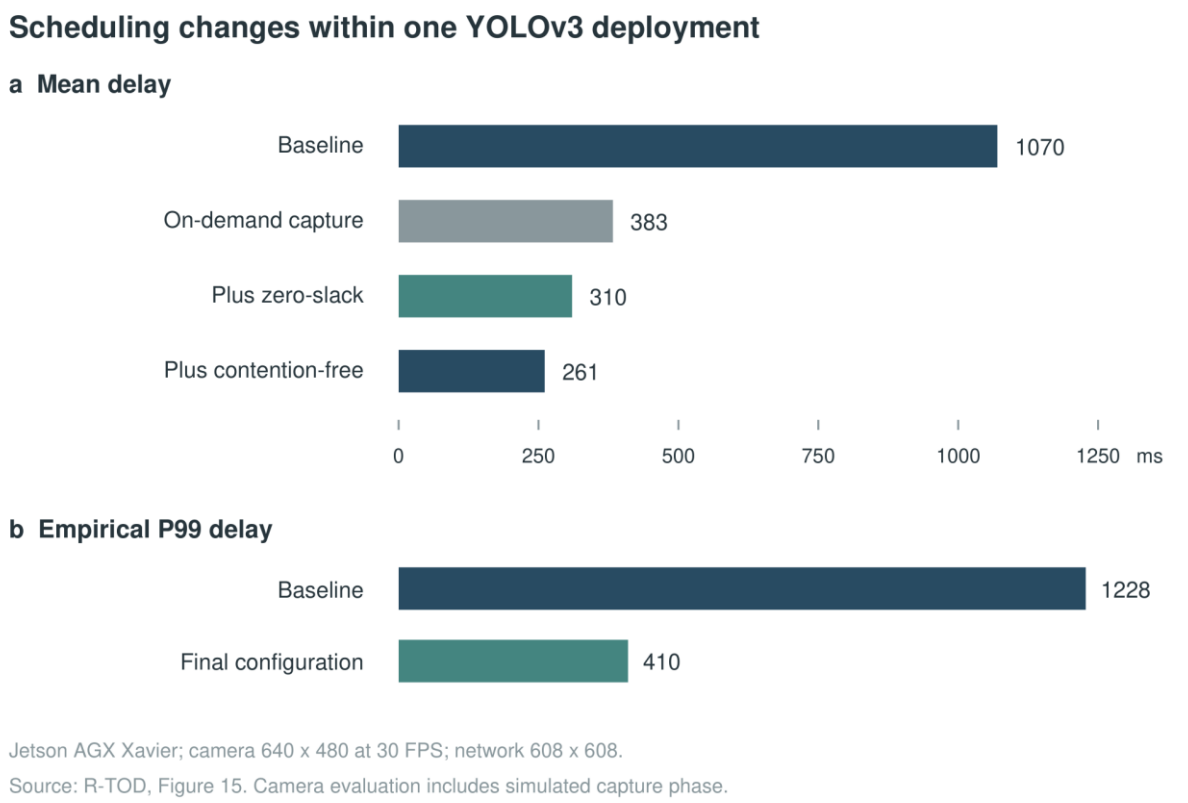


Figure 3. R-TOD within-study timing changes for YOLOv3 on Jetson AGX Xavier, using [7], introduction and Figure 15. The top panel shows successive mean-delay configurations. The lower panel contrasts baseline and final empirical P99 values. The source camera-subsystem evaluation includes a simulated capture-phase component. Bars do not represent worst-case guarantees.

4.2. Transport cost depends on what is transported

Transport includes several distinct operations: sensor-to-memory delivery, host-device tensor copies, transfers between processing stages, and network transmission. Their payloads, buffering rules, and clocks differ. Qasaimeh et al. explicitly separate compute-oriented and system-oriented measurements [17]. Their passthrough data-mover experiment reports 6.945 ms on a field-programmable gate array (FPGA) and 1.298 ms on GPU for a 1080p image. These are memory-movement measurements in their implementations, not camera readout times or universal bus latencies.

The distinction matters for FPGA comparisons. A streaming design can fuse operations while avoiding intermediate round trips to external memory. That benefit depends on how the stages are composed and on the input/output interfaces. An isolated accelerator kernel may therefore have a different advantage from the complete pipeline. Likewise, an integrated CPU-GPU system can avoid a discrete-device copy while still contending for memory bandwidth. Hardware category alone does not specify the data path.

Input resolution has at least two meanings: sensor output dimensions and network tensor dimensions. Reducing only the tensor size can leave readout, decoding, and host traffic unchanged. Cropping or reducing camera output can alter those costs but also changes field of view, small-object evidence, and exposure behavior. A resolution comparison should identify which image is resized, where resizing occurs, and whether the task-quality target is preserved. Platform-aware architecture search supports measuring realized latency on the target device rather than inferring it from operation counts [18].

4.3. Preprocessing and output work can change the bottleneck

Preprocessing may include decoding, color conversion, resizing, normalization, and tensor layout conversion. Some operations run on the host; others are fused into an accelerator graph. Postprocessing can include suppression, coordinate remapping, tracking, segmentation-mask reconstruction, or output serialization. A timer placed around a model API may include different subsets across implementations. Removing non-maximum suppression can improve detector deployment, as RT-DETR and YOLOv10

illustrate [19, 20]. Their architectural use of end-to-end does not imply that physical camera acquisition was timed.

Multi-stage perception adds a workload dimension that a single input size cannot capture. In the date-palm detect-to-cut pipeline, pose estimation runs on each selected region of interest (ROI) [21]. Its multi-region timing breakdown assigns most processing to the pose stage, while crop resizing and final remapping remain smaller components. Table 3 preserves the reported average together with the one-region case. The appropriate comparison unit includes the number and distribution of selected regions, rather than only frames or detector parameters.

The same source reports 380.34 ms mean total latency and 4.00 mean effective FPS for the multi-region evaluation. These values are not reciprocals. Averaging per-scene reciprocal rates could contribute to this difference, but the aggregate reporting does not establish a shared run-level denominator. We therefore retain both source labels without deriving a cycle interval. The displayed stage means also sum to 380.35 ms, a 0.01 ms rounding difference from the total. Neither discrepancy justifies silently changing the published values.

Output requirements can alter the bottleneck independently of the neural model. Podbucki and Szalwach compare saving numerical results with additionally rendering and encoding annotated video [22]. In their prerecorded-video evaluation, output generation reduces pipeline throughput while inference timing remains similar. This supports measuring the output actually required by the application. It does not justify treating recording throughput as camera-to-controller latency, or treating a debug visualization cost as mandatory for every robot.

Table 3. Host processing and hardware evidence. NR means not reported in the inspected passages. ROI means region of interest; NPU means neural processing unit. FP16 denotes 16-bit floating point; CSV denotes comma-separated values.

Source and workload	Deployment and reported result	Comparison restriction
Qasaimeh et al. [17] 1080p passthrough data movement	ZCU102 FPGA / Jetson TX2 GPU 6.945 / 1.298 ms (Table 2)	Memory data-mover interval; neither sensing nor complete neural pipeline
Maghzaoui et al. [21] Detect-to-cut processing; 640 × 640 stage inputs; batch 1	Orin NX 16 GB; 25 W, locked clocks; FP16 TensorRT Overall 380.34 ms, 4.00 FPS; single ROI 151.03 ms, 6.65 FPS	Host processing, not acquisition timing; ROI mix changes work; mean latency/FPS aggregation unresolved
Podbucki and Szalwach [22] Point localization and segmentation on recorded video	Raspberry Pi 5 + Hailo-8 INT8 / AGX Orin FP16 CSV-only versus CSV plus annotated-video modes	Compare within matched model and mode; source video 1920 × 1080 differs from model input; no exposure endpoint
Kong et al. [23] YOLO v5/v8/v11 model scales; 640 × 640 comparison	RV1106 / RK3568 / RK3588 NPUs Nano-model latency ranges 68-97 / 34-79 / 21-30 ms	Source-defined NPU-runtime processing interval; ranges span models; acquisition and exact substage inclusion unresolved

4.4. Precision, runtime, and hardware form one operating point

A hardware label is insufficient for comparison because model conversion and runtime determine which operations execute on the accelerator. Precision can change memory traffic, kernel selection, calibration error, and fallback behavior. Comparing an INT8 NPU implementation with an FP16 GPU implementation is a deployment comparison, but it does not isolate accelerator architecture. Quality should be evaluated after conversion, and any unsupported operations or host fallback should be disclosed.

Kong et al. evaluate low-compute Rockchip systems across model scales, resolutions, NPU configurations, and contention conditions [23]. Their results show that nominal processing capacity does not translate proportionally into latency improvements. The source defines latency at the NPU-runtime level; its real-time thresholds do not establish robot-specific response deadlines. We use the study to support conditional hardware interpretation, while leaving acquisition and preprocessing coverage unresolved. Its methods and

discussion also describe resolution ranges differently, so the table confines its numerical example to the explicitly shared 640 × 640 setting.

Thermal state and concurrent work belong to this operating point. A short warmed inference loop can characterize steady engine execution without representing camera decoding, navigation, logging, or a second neural model. Conversely, a complete robot workload can slow inference through shared-resource contention. The useful question is which resources are shared under the intended workload, rather than whether a platform is generally faster. Reviews of efficient inference and measured embedded pipelines support this systems interpretation [5, 17].

4.5. Offloading and tail behavior require distinct endpoints

Offloading is favorable for latency only when its additional costs are smaller than the compute time saved at a matched output-quality target. A schematic condition is

$$T_{encode} + T_{uplink} + T_{remote\,queue} + T_{remote\,compute} + T_{downlink} + T_{decode} < T_{local} \quad (4)$$

All terms in Equation (4) must refer to the same input observation and usable output. Capture and common local work cancel only if they are truly identical. Connection setup, retransmission, jitter buffering, and service timeouts may change the condition. An edge location can shorten one path while leaving generation or serialization dominant. The inequality is a decision aid, not an empirical prediction of an unmeasured network.

FogROS2 provides an instructive boundary distinction [24]. Its video-transport experiment measures publication-to-acknowledgement round-trip time, without a visual inference task at the remote node. Its ORB-SLAM2 evaluation instead measures the round trip through a processing node. The strong compression improvement in the first experiment does not imply the same percentage improvement in SLAM, grasping, or robot control. Table 4 keeps these results separate.

A mean reduction also leaves reliability unresolved. FogROS2-PLR replicates requests across network interfaces and cloud services, evaluating tail latency as well as a physical human-tracking robot [25]. The inspected author manuscript reports improvements in mean and P99 timing on Stretch 3. This is evidence that service diversity can improve the measured distribution. It does not demonstrate independence under every outage, nor does the P99 result bound the unobserved tail.

Table 4. Networked and semantic-response evidence. RTT means round-trip time. SLAM means simultaneous localization and mapping. P99 denotes the empirical 99th percentile.

Source and endpoint	Published observation	Permitted interpretation
FogROS2 transport [24] Publish image → remote acknowledgement	Intel NUC / AWS; 10 Mbps link; 3000 images Uncompressed 1401 ms; H.264 38 ms RTT	Transport configuration effect; no inference workload in this particular test
FogROS2 ORB-SLAM2 [24] Per-frame processing-node RTT	TUM fr1/xyz; 36-core cloud FogROS2 image compression 0.82 s; H.264 0.24 s	Application RTT; different work and endpoint from the transport test
FogROS2-PLR [25] Cloud-connected human tracking	Physical Stretch 3; replicated network/service paths Reported mean reduction 36%; P99 reduction 33%	Within-study responsiveness benefit; does not imply a hard deadline guarantee
Ahmad et al. [26] Frame/query → final textual response	Unitree G1; Llama-3.2-11B-Vision Edge 1600.03 ms; cloud 1685.20 ms	Preprint; capture-origin and query-origin wording differ; not a per-frame control-loop deadline

4.6. Boundary cases for embodied intelligence

The visual-language deployment in Table 4 broadens the meaning of usable result [26]. A final natural-language answer includes autoregressive generation, whereas a bounding box or pose estimate may be usable earlier. The source defines latency from frame capture to final response, then describes live interaction timing from query initiation. Without a reconciled origin and output-length specification, those descriptions leave the common interval uncertain. Its reported edge-cloud difference is retained as a source result, with no claim about navigation safety or actuation responsiveness.

For vision-language systems, useful reporting would distinguish frame age at query initiation, time to first usable token or structured decision, and time to completed response. Generation length and stopping criteria should accompany the statistic. For action-generating systems, a separate action-consumption boundary would be required. These are extensions of the measurement logic, not evidence that a particular foundation model meets a robot's control deadline.

Event cameras challenge the assumption that one frame defines one observation. Event-based vision offers asynchronous sensing and low sensor latency, but algorithms may accumulate events into windows before inference [27]. Window duration, timestamp assignment, and processing delay must then be reported separately. A short sensor response or fast inference over an event tensor does not automatically establish a short physical-event-to-result interval. Quantitative event/frame hardware ranking is outside the present evidence core.

5. A Four Gate Procedure for Comparing Reports

A numerical comparison is admissible only for the claim whose necessary conditions are satisfied. Figure 4 summarizes four gates. The first checks whether both reports time the same start event, end event, and included stages. If not, they can support a qualitative mechanism comparison but not a common latency ranking. A source calling both values end-to-end does not satisfy this gate by itself.

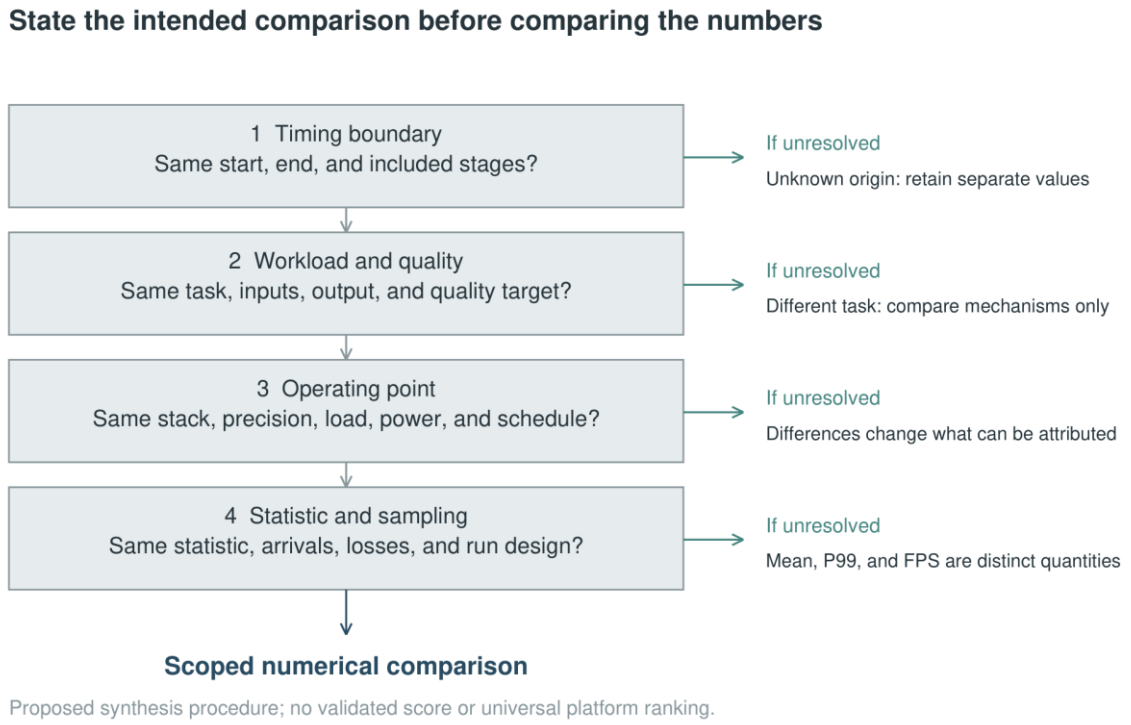


Figure 4. Proposed comparison procedure. A failed or unresolved gate restricts the numerical conclusion; it does not invalidate the original study. A claim can proceed after explicitly conditioning on a differing factor, such as comparing complete deployment stacks rather than isolating hardware.

The second gate checks workload and output semantics. Tasks, input content, sensor and tensor resolutions, region counts, output requirements, and quality targets must be aligned with the intended conclusion. A

detector benchmark and a final language response fail this gate for a common control-response ranking. Two multi-region pipelines can also fail despite matching model names and tensor dimensions.

The third gate checks operating conditions: runtime, precision, power mode, clocks, batching, concurrency, queue policy, network conditions, and thermal regime. Differences need not forbid every comparison. They change the estimand. For example, a matched-task comparison of INT8 and FP16 stacks can answer which measured deployment meets a quality and latency target. It cannot attribute the difference solely to GPU versus NPU hardware.

The fourth gate checks the statistic and observation process. Mean, median, P99, sample maximum, reciprocal throughput, and time-to-first-response are not substitutes. Run length, warm-up, repeated runs, request arrival policy, frame loss, and timeout handling determine what entered the distribution. If these are unresolved, report the source values separately. Adding an uncertainty bar invented from another paper would not repair the mismatch.

The procedure has no compensation mechanism: complete hardware metadata cannot repair an unknown timer origin. It also avoids an automatic preference for larger boundary classes. For a compiler claim, matched B0 measurements may be sufficient. For a freshness claim at a robot consumer, matched acquisition and delivery semantics are required. The output is a scoped statement of comparability, with named unresolved fields, rather than a universal benchmark score.

6. Reporting and Reuse in Robotic Deployments

6.1. A minimum timing record

The selected evidence supports a small set of reporting requirements that should accompany any deployment latency claim. Table 5 gives a reusable specification. It is proposed for future studies and evidence extraction; no new traces were collected to validate it here. Its purpose is to let a reader reproduce the measurement interval and determine which comparison is justified.

Table 5. Proposed minimum reporting record for a visual deployment. Include NR explicitly when retrospective extraction cannot recover a field.

Field group	Minimum content	Why it matters
Observation provenance	Frame identifier; sensor dimensions; tensor dimensions; exposure/readout origin; dropped-frame policy	Connects result age to a physical observation
Timed interval	Named start and completion events; included stages; CPU/GPU synchronization; output-ready definition	Prevents comparing unlike meanings of latency
Clock relationship	Clock type/domain; synchronization method and uncertainty; one-way versus RTT	Exposes offsets and unmeasured return paths
Execution stack	Model/checkpoint; precision; runtime/compiler versions; host fallback; batch and concurrency	Defines the computation that actually ran
Deployment conditions	Hardware; power/clock settings; cooling; sustained duration; concurrent tasks; network/codec	Separates configuration effects from platform claims
Scheduling and buffering	Queue capacities and policy; frame selection; sensor/consumer rates; middleware quality of service	Explains stale results and overload behavior
Statistics and failures	Sample counts; repeats; warm-up; mean/median and relevant tails; timeout/loss counts; deadline if tested	Defines the observed distribution and its censoring
Task quality and artifacts	Quality metric after conversion/compression; scene workload; scripts/trace schema; source version	Prevents speed improvements from concealing task changes

A useful record stores per-frame acquisition time, arrival at each queue, dispatch and completion events, and arrival at the result consumer. Stage summaries should retain their relationship to the total. This enables a reader to identify gaps, overlap, and double counting. If an optical ground-truth event is unavailable, the paper should identify the earliest observable software event rather than imply that exposure or physical appearance was measured.

ROS 2 tracing can recover internal execution information, and message-flow analysis can link messages across distributed computation graphs [28, 29]. These facilities support diagnosis of callback, executor, and communication behavior. They do not automatically validate a sensor timestamp or synchronize two hosts. A study should retain message lineage and document any many-to-one associations, such as a detection selected from several frames or a fused result derived from different sensors.

Middleware quality-of-service settings should be part of the record. ROS 2 provides history, depth, reliability, deadline, and lifespan policies, with sensor-oriented settings favoring timely delivery under their documented trade-offs [30]. A configured middleware deadline is not proof that an image reached an application before a physical control deadline. Similarly, reducing history depth can shorten a backlog while dropping observations. The evaluation must count those drops and test the temporal assumptions of downstream tracking or fusion.

6.2. An operational measurement sequence

First, define the deployment question before selecting the timer. For a mobile robot's obstacle detector, this could be acquisition-to-result delay at the navigation input under a stated workload. For compiler tuning, it could instead be batch-one engine execution at fixed quality. The sensor sampling rate and the consumer deadline should be recorded as separate quantities. A nominal 30 FPS camera does not imply that a 33.3 ms perception budget is either necessary or sufficient for the robot.

Second, measure a complete interval and then instrument its stages. An engine-only loop can isolate numerical or compiler changes; a complete host pipeline can expose conversion and output overhead. A live acquisition-to-consumer trace can reveal buffering and contention. These measurements serve different purposes and should retain separate labels. A replay experiment should disclose pacing, original timestamps, and whether disk I/O is included, because accelerated playback changes the arrival process.

Third, evaluate the intended operating regime. Warm-up should be separated from steady-state results, while cold-start or intermittent-use delay should be retained if it matters to the robot. Sustained runs should record temperatures, clock behavior, background tasks, and arrival rates. Queue occupancy and frame-age distributions are particularly useful under overload. A latest-frame strategy should be evaluated for missed transient events as well as reduced age.

Fourth, summarize both responsiveness and task performance. Report the relevant latency distribution, sustained output rate, dropped or timed-out observations, and quality after precision conversion or compression. If a deadline is part of the claim, provide its definition and denominator. Raw traces need not include private images: identifiers, timestamps, workload descriptors, configuration files, and summary scripts can often support timing audit without releasing visual content.

6.3. Choosing an optimization from evidence

Optimization should follow the component that limits the intended interval. When driver backlog dominates, queue and dispatch changes are plausible candidates. When the front-end aggregate dominates, exposure, readout, pixel format, and preprocessing placement deserve inspection. When region-dependent work dominates, region selection, batching policy, or stage sharing may matter. These alternatives need task-quality checks because missing an image or candidate is not equivalent to processing it faster.

The limiting-component principle can be expressed with a conditional speedup calculation. Let f be the fraction of the measured interval attributable to a component, and s its isolated speedup. If all other costs remain fixed, the total speedup is

$$S_{total} = \frac{1}{(1 - f) + f / s} \quad (5)$$

Equation (5) explains the small theoretical gain from accelerating TinyTracker inference alone. Its assumptions can fail if overlap, scheduling, power management, or contention changes. A faster kernel can release memory bandwidth for another stage, while a more aggressive accelerator mode can worsen thermal behavior. The equation therefore bounds an unchanged-cost scenario; it does not replace measurement of the revised pipeline.

7. Discussion and Research Priorities

7.1. What the evidence changes

The evidence favors a conditional view of latency optimization. Published deployments expose different limiting stages because they measure different paths under different workloads. Queue control, front-end design, region-dependent execution, and output generation each have direct examples in the core. None should be declared the dominant cost of robotic edge vision in general. The transferable conclusion is to preserve measurement boundaries while locating the bottleneck of the intended result path.

This conclusion qualifies common hardware comparisons. Engine rankings can guide a component choice when the deployment actually remains compute-limited. They become less informative when data movement, waiting, or output handling dominates. An apparent disagreement can reflect different measurement targets rather than contradictory accelerator behavior. Keeping the boundary and operating point visible permits useful comparisons without forcing unlike studies into a single leaderboard.

The review also changes what counts as missing evidence. An unspecified clock origin is not merely a formatting omission when the claim concerns acquisition-to-result delay. An unspecified region mix prevents extrapolation of a multi-stage mean to a crowded scene. Excluded timeouts prevent interpreting a reported percentile as service reliability. These gaps directly delimit the claims supported by an otherwise valid experiment.

7.2. Limits of the synthesis

The evidence core is purposively selected and small relative to the broader embedded-vision literature. Public accessibility, English-language discovery, search-engine ranking, and recognizable system names may bias the selection. It includes related Darknet studies rather than independent replications, and several sources evaluate transferable subsystems rather than complete robots. Accordingly, the extraction cannot estimate how frequently the entire field omits a timing component or determine the prevalence of a particular bottleneck.

Most quantitative conclusions remain within-study observations. Hardware, models, software stacks, and workloads differ too much to estimate a pooled speedup. Published aggregate results cannot recover frame-level distributions, shared-stage correlations, or sensor-clock uncertainty. Reporting ambiguities are therefore preserved rather than resolved by inference. Even a fully specified source result remains conditional on its tested scenes and operating duration.

The latest vision-language example is a preprint, and its timer-origin inconsistency limits its use beyond demonstrating the need for output semantics. Event-based sensing, multi-camera fusion, SLAM internals, action-generation models, and actuator dynamics are not exhaustively reviewed. The four classes and comparison gates have analytical utility, but have not been validated through inter-rater coding or a prospective benchmark. They should be tested as reporting aids rather than treated as certification criteria.

A further limitation concerns physical meaning. An accurate acquisition timestamp can still represent an extended exposure or a rolling readout rather than an instantaneous world state. A usable result can also contain predictions about a future state. Observation age, model prediction horizon, and eventual control effectiveness should therefore remain distinct. This review does not infer collision avoidance, stability, or safe operating speed from latency alone.

7.3. Experiments that would resolve the remaining uncertainty

A useful next benchmark would pair live sensor-to-consumer traces with matched engine and host-pipeline measurements. It should hold task quality constant while varying queue policy, preprocessing location, and

resource contention. An independently timestamped visual stimulus would help distinguish modeled capture phase from measured physical response. The necessary output is a causal trace with a documented clock relationship, not simply another average FPS table.

Resolution studies should vary camera output and network tensor size independently. Reporting object size, detection quality, and transmitted bytes would reveal whether a latency gain comes from reduced sensor traffic, reduced network work, or a changed task. Multi-region and multi-camera studies should publish workload-conditioned distributions. This would test whether an average deployment result remains relevant when scene complexity or synchronization demand increases.

For offloaded perception, future evaluations should retain timeouts and handovers while comparing complete result paths at matched quality. Independent network paths should be assessed under correlated failures before reliability extrapolations. For vision-language perception, input-frame age and first-usable-output timing should accompany final-response timing. For all three cases, the research priority is to explain distributional change with observable events and task consequences.

8. Conclusions

End-to-end latency in robotic edge vision is meaningful only relative to a named observation, completion event, and deployment condition. Public results show substantial effects from acquisition, queues, transport, region-dependent processing, and output handling, alongside neural execution. Their diversity supports a mechanism-based synthesis, while their measurement differences limit a common numerical ranking.

This review contributes four descriptive boundary classes, a traceable deployment evidence table, a four-gate comparison procedure, and a minimum timing record. Together they connect each reported number to the deployment claim it can support. The practical recommendation is to measure the path to the robot's usable result, retain stage and clock semantics, and evaluate freshness and quality alongside throughput. These practices improve interpretability; they do not establish a universal accelerator choice or guarantee a control deadline.

Supplementary Materials

The following supplementary materials accompany the review: Table S1, the deployment evidence extraction table; Table S2, the search-query log; Table S3, bibliographic verification; and Supplementary Note S1, extraction decisions, arithmetic checks, and source-access limitations. No unpublished deployment measurements were generated. Public source PDFs are retained for private audit and are not redistributed as supplementary material.

Author Contributions

[AUTHOR TO COMPLETE: assign only contributions actually performed and verified by each named human author, using the journal's CRediT roles.]

Funding

[AUTHOR TO COMPLETE: confirm the actual funding information and grant identifiers, or confirm the absence of external funding.]

Institutional Review Board Statement

Not applicable. This review analyzes published reports and public documentation and involved no recruitment or new human or animal experiments.

Informed Consent Statement

Not applicable.

Data Availability Statement

The extracted deployment evidence, search log, bibliographic verification, and extraction notes are provided in the Supplementary Materials. Original deployment results remain available from the cited publications and project documentation. No new experimental data were collected.

Acknowledgments

[AUTHOR TO COMPLETE: acknowledgments, if applicable.]

Conflicts of Interest

[AUTHOR TO COMPLETE: disclose actual conflicts of interest or confirm their absence.]

References

1. Chaumette, F.; Hutchinson, S. Visual servo control. I. Basic approaches. IEEE Robotics & Automation Magazine 2006, 13, 82-90. https://doi.org/10.1109/mra.2006.250573.

2. Li, M.; Wang, Y.X.; Ramanan, D. Towards Streaming Perception. Lecture Notes in Computer Science 2020, 473-488. https://doi.org/10.1007/978-3-030-58536-5_28.

3. Zhou, Z.; Chen, X.; Li, E.; Zeng, L.; Luo, K.; Zhang, J. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. Proceedings of the IEEE 2019, 107, 1738-1762. https://doi.org/10.1109/jproc.2019.2918951.

4. Sze, V.; Chen, Y.H.; Yang, T.J.; Emer, J.S. Efficient Processing of Deep Neural Networks: A Tutorial and Survey. Proceedings of the IEEE 2017, 105, 2295-2329. https://doi.org/10.1109/jproc.2017.2761740.

5. Shuvo, M.M.H.; Islam, S.K.; Cheng, J.; Morshed, B.I. Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices: A Review. Proceedings of the IEEE 2023, 111, 42-91. https://doi.org/10.1109/jproc.2022.3226481.

6. Jeon, W.; Kang, K.; Kim, J.C. End-to-End Delay Analysis and Optimization of Object Detection Module for Autonomous Driving. Machine Learning for Autonomous Driving Workshop at NeurIPS 2019 2019, 1-9. Available online: https://ml4ad.github.io/files/papers/End-to-End%20Delay%20Analysis%20and%20Optimization%20of%20Object%20Detection%20Module%20for%20Autonomous%20Driving.pdf (accessed on 19 October 2026).

7. Jang, W.; Jeong, H.; Kang, K.; Dutt, N.; Kim, J.C. R-TOD: Real-Time Object Detector with Minimized End-to-End Delay for Autonomous Driving. 2020 IEEE Real-Time Systems Symposium (RTSS) 2020, 191-204. https://doi.org/10.1109/rtss49844.2020.00027.

8. Elhanashi, A.; Saponara, S. Machine Vision in Real-Time Environments: A Narrative Review. Electronics 2026, 15, 4568. https://doi.org/10.3390/electronics15194568.

9. Cordova-Cardenas, R.; Amor, D.; Gutiérrez, Á. Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems. Electronics 2025, 14, 4877. https://doi.org/10.3390/electronics14244877.

10. Varghese, B.; Wang, N.; Bermbach, D.; Hong, C.H.; Lara, E.D.; Shi, W.; Stewart, C. A Survey on Edge Performance Benchmarking. ACM Computing Surveys 2022, 54, 1-33. https://doi.org/10.1145/3444692.

11. The Linux Kernel Documentation. Buffers. Video for Linux Two API Specification. Available online: https://docs.kernel.org/userspace-api/media/v4l/buffer.html (accessed on 19 October 2026).

12. Open Robotics. Image.msg. ROS 2 common_interfaces repository, rolling branch. Available online: https://github.com/ros2/common_interfaces/blob/rolling/sensor_msgs/msg/Image.msg (accessed on 19 October 2026).

13. NVIDIA. Performance Benchmarking. NVIDIA TensorRT Documentation, snapshot accessed 19 October 2026. Available online: https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/benchmarking.html (accessed on 19 October 2026).

14. Yates, R.D.; Sun, Y.; Brown, D.R.; Kaul, S.K.; Modiano, E.; Ulukus, S. Age of Information: An Introduction and Survey. IEEE Journal on Selected Areas in Communications 2021, 39, 1183-1210. https://doi.org/10.1109/jsac.2021.3065072.

15. Reddi, V.J.; Cheng, C.; Kanter, D.; Mattson, P.; Schmuelling, G.; Wu, C.J.; Anderson, B.; Breughe, M.; Charlebois, M.; Chou, W.; et al. MLPerf Inference Benchmark. 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA) 2020, 446-459. https://doi.org/10.1109/isca45697.2020.00045.

16. Bonazzi, P.; Rüegg, T.; Bian, S.; Li, Y.; Magno, M. TinyTracker: Ultra-Fast and Ultra-Low-Power Edge Vision In-Sensor for Gaze Estimation. 2023 IEEE SENSORS 2023, 1-4. https://doi.org/10.1109/sensors56945.2023.10325167.

17. Qasaimeh, M.; Denolf, K.; Khodamoradi, A.; Blott, M.; Lo, J.; Halder, L.; Vissers, K.; Zambreno, J.; Jones, P.H. Benchmarking vision kernels and neural network inference accelerators on embedded platforms. Journal of Systems Architecture 2021, 113, 101896. https://doi.org/10.1016/j.sysarc.2020.101896.

18. Tan, M.; Chen, B.; Pang, R.; Vasudevan, V.; Sandler, M.; Howard, A.; Le, Q.V. MnasNet: Platform-Aware Neural Architecture Search for Mobile. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2019, 2815-2823. https://doi.org/10.1109/cvpr.2019.00293.

19. Zhao, Y.; Lv, W.; Xu, S.; Wei, J.; Wang, G.; Dang, Q.; Liu, Y.; Chen, J. DETRs Beat YOLOs on Real-time Object Detection. 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2024, 16965-16974. https://doi.org/10.1109/cvpr52733.2024.01605.

20. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G. YOLOv10: Real-Time End-to-End Object Detection. Advances in Neural Information Processing Systems 2024, 37, 107984-108011. https://doi.org/10.52202/079017-3429.

21. Maghzaoui, A.; Riahi, S.; Aridhi, E.; Ben Yahia, S.; Mami, A. A Two-Stage Embedded Detect-to-Cut Vision Pipeline for Date Palm Branch Analysis on Jetson Orin NX. Computers 2026, 15, 589. https://doi.org/10.3390/computers15090589.

22. Podbucki, K.; Szalwach, B.o. Cost-Effective Edge AI: Hailo-8 Powered Raspberry Pi vs. NVIDIA Jetson AGX Orin in Airport Infrastructure Monitoring. Electronics 2026, 15, 3774. https://doi.org/10.3390/electronics15173774.

23. Kong, C.; Li, F.; Yan, X.; Yang, J.; Mo, P.; Luo, Q.; Mao, R. Object detection on low-compute edge SoCs: a reproducible benchmark and deployment guidelines. Scientific Reports 2026, 16, 5875. https://doi.org/10.1038/s41598-026-36862-y.

24. Ichnowski, J.; Chen, K.; Dharmarajan, K.; Adebola, S.; Danielczuk, M.; Mayoral-Vilches, V.; Jha, N.; Zhan, H.; Llontop, E.; Xu, D.; et al. FogROS2: An Adaptive Platform for Cloud and Fog Robotics Using ROS 2. 2023 IEEE International Conference on Robotics and Automation (ICRA) 2023, 5493-5500. https://doi.org/10.1109/icra48891.2023.10161307.

25. Chen, K.; Tian, N.; Juette, C.; Qiu, T.; Ren, L.; Kubiatowicz, J.; Goldberg, K. FogROS2-PLR: Probabilistic Latency-Reliability for Cloud Robotics. 2025 IEEE International Conference on Robotics and Automation (ICRA) 2025, 16290-16297. https://doi.org/10.1109/icra55743.2025.11127588.

26. Ahmad, S.; Hafeez, M.; Zaidi, S.A.R. Vision-Language Models on the Edge for Real-Time Robotic Perception. arXiv preprint 2026. https://doi.org/10.48550/arXiv.2601.14921. Preprint.

27. Gallego, G.; Delbruck, T.; Orchard, G.; Bartolozzi, C.; Taba, B.; Censi, A.; Leutenegger, S.; Davison, A.J.; Conradt, J.; Daniilidis, K.; et al. Event-Based Vision: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence 2022, 44, 154-180. https://doi.org/10.1109/tpami.2020.3008413.

28. Bedard, C.; Lutkebohle, I.; Dagenais, M. ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2. IEEE Robotics and Automation Letters 2022, 7, 6511-6518. https://doi.org/10.1109/lra.2022.3174346.

29. Bédard, C.; Lajoie, P.Y.; Beltrame, G.; Dagenais, M. Message flow analysis with complex causal links for distributed ROS 2 systems. Robotics and Autonomous Systems 2023, 161, 104361. https://doi.org/10.1016/j.robot.2022.104361.

30. Open Robotics. Quality of Service Settings. ROS 2 Jazzy Documentation, source repository. Available online: https://github.com/ros2/ros2_documentation/blob/jazzy/source/Concepts/Intermediate/About-Quality-of-Service-Settings.rst (accessed on 19 October 2026).